Simplified Quantization for Edge-Ready Language Models: Salience-Guided Mixed-Precision Post-Training Quantization at 1.61 Bits

Anshul Dani
A20580060
adani2@hawk.illinoistech.edu

Rohit Lahori
A20582911
rlahori@hawk.illinoistech.edu

Illinois Institute of Technology — CS595-2 — May 2026

Abstract

Large language models are too large for edge devices, and uniform post-training quantization (PTQ) below 2 bits/weight collapses model accuracy. We present **Salient-Quant** (SMQ), a three-phase PTQ framework that allocates a mixed $\{1, 2, 4\}$-bit budget across weights according to a five-metric salience ensemble (magnitude L1/L2, gradient, Hessian-diagonal Fisher, activation-aware scaling), and applies row-coherent block-wise quantization with closed-form OLS scale refinement. On **GPT-2 medium (345M)**, SMQ achieves WikiText-2 PPL of **541.05** at 1.61 bits/weight — **2,683**× better than Uniform INT2 (1.45M PPL) at higher compression (9.94× vs. 7.97×). On **LLaMA-3.2-1B**, SMQ reaches **35,831** PPL at 1.61 bits (10.4× compression) — 4.6× better than Uniform INT2 and 2.8× better than BitNet at the same scale. A six-axis ablation study isolates each design choice, surfacing three findings that contradict the project's design assumptions: (i) magnitude alone is the strongest single salience signal, (ii) symmetric 2-bit beats asymmetric on LLaMA-1B by 9.6×, and (iii) the dominant accuracy lever is *spatial coherence* of block-wise scales — a row-wise quantization fix alone took GPT-2 medium PPL from 12,173 to 541 (22×), eclipsing every salience metric. We also document the seven engineering fixes needed to fit the pipeline within a 16 GB Colab T4.

1 Motivation and Problem Statement

State-of-the-art LLMs require gigabytes of memory: LLaMA-3.2-1B at FP16 occupies 2.5 GB, a 7B model needs >13 GB. Edge devices typically expose under 256 MB of usable RAM, so deployment requires 10× or greater compression. Post-training quantization (PTQ) is the preferred mechanism — no fine-tuning, generalizes across model families — but sub-2-bit PTQ is brittle. Uniform INT2 inflates GPT-2 medium WikiText-2 perplexity from 21.64 to 1.45×10^6 (a 67,000× degradation); BitNet's ternary scheme is no better as a PTQ (1.56×10^6 PPL) because it was designed for training-time integration. The 4-bit regime (GPTQ, AWQ) is well-studied but only delivers 4× compression. The interesting 2-bit regime is where uniform methods break.

This project asks: can a non-uniform allocation of $\{1, 2, 4\}$ bits, guided by a per-weight importance score, achieve a sub-2-bit average while preserving usable accuracy? The answer addresses both an algorithmic question (what does "important" mean for weights?) and a systems question (can the resulting pipeline fit and run within a commodity 16 GB GPU?).

Contributions. (i) A vectorized 5-metric salience ensemble computed in a single calibration pass. (ii) A greedy bit allocator that distributes $\{1, 2, 4\}$-bit budgets across all parameters in a single `torch.topk` call (0.45 s for 345M weights). (iii) A row-coherent block-wise quantizer with closed-form OLS scale refinement, achieving **541 PPL** at 1.61 bits on GPT-2 medium. (iv) A six-axis

ablation study on LLaMA-3.2-1B with full JSON outputs released. (v) Documented engineering: seven memory and numerical-stability fixes that enabled the pipeline to run on a 16 GB T4, including a row-coherence fix that alone gave 22× PPL.

2 Background and Related Work

Uniform PTQ. GPTQ [1] quantizes LLMs to 4-bit via Hessian-based optimal rounding; AWQ [2] protects salient *channels* at higher precision; ZeroQuant [8] adds hardware-aware kernels. All target 4-bit; none cross the 2-bit barrier. **Sub-2-bit.** BitNet [3] uses ternary weights at 1.58 bits but requires training-time integration, collapsing as PTQ. LLM.int8() [5] mixes INT8/FP16 outliers but stops at 8-bit. **Salience-aware.** SmoothQuant [6] transfers difficulty between activations and weights; OBD [4] and OBS [7] use Hessian information for pruning. We extend this line by combining five complementary metrics into a per-weight ensemble used for *bit allocation* rather than pruning. **Datasets and tools.** Calibration: C4 validation [10]. Evaluation: WikiText-2 [12] and Penn Treebank. Models: GPT-2 [11], LLaMA-3.2-1B [9]. Implementation: PyTorch 2.x, HuggingFace Transformers, Colab T4.

3 Methodology

SMQ runs three sequential phases on a pre-trained model with access to a small calibration set; no gradient updates are applied to model parameters.

Phase 1 – Salience scoring. Each weight gets a score from five min-max-normalized-per-layer metrics combined as $s_{\text{ens}}(w) = \alpha_1 \hat{s}_{\text{L1}} + \alpha_2 \hat{s}_{\text{L2}} + \alpha_3 \hat{s}_{\text{grad}} + \alpha_4 \hat{s}_{\text{H}} + \alpha_5 \hat{s}_{\text{act}}$. The metrics are $s_{\text{L1}} = |w|$, $s_{\text{L2}} = w^2$, $s_{\text{grad}} = |w \cdot \nabla_w \mathcal{L}|$ [4], $s_{\text{H}} = \mathbb{E}[(\nabla_w \mathcal{L})^2]$ (empirical Fisher diagonal [7]), and $s_{\text{act}}(w_{ij}) = |w_{ij}| \cdot \text{std}(x_j)$ following SmoothQuant. Default $\boldsymbol{\alpha} = (0.15, 0.15, 0.25, 0.25, 0.20)$; Section 6 shows magnitude-heavy weighting empirically dominates at 1.61 b on LLaMA-3.2-1B.

Phase 2 – Greedy bit allocation. Given $B_{\text{target}} = 1.61$ and bit choices $\mathcal{B} = \{1, 2, 4\}$: initialize $b_i = 1$ for all weights, set budget $= (B_{\text{target}} - 1) \cdot N$, then for each $b \in \{2, 4\}$ ascending, take the top-k weights by salience ($k = \lfloor \text{budget}/(b-1) \rfloor$) and upgrade them to b bits, decrementing the budget. The top-k step is one GPU `torch.topk` call over all weights concatenated; allocation completes in 0.45 s for 85M-quantizable-weight GPT-2 medium.

Phase 3 – Row-coherent mixed-precision quantization. Each weight is quantized at its assigned bit-width: 1-bit uses $q = \text{sign}(w) \cdot \overline{|w|}$ per block (BitNet-style); 2-bit uses 4 symmetric levels $\{\pm 0.5s, \pm 1.5s\}$; 4-bit uses INT4 symmetric (15 levels). For 2- and 4-bit, given fixed level codes $\mathbf{L}$, the closed-form scale minimizing $\|\mathbf{W} - s\mathbf{L}\|_2^2$ is $s^* = (\mathbf{W} \cdot \mathbf{L})/(\mathbf{L} \cdot \mathbf{L})$, replacing a per-block Adam loop with two vectorized dot products.

The row-coherence fix. Our first implementation flattened each weight matrix into a 1D array before applying the boolean salience mask, so block boundaries cut across unrelated rows — the per-block scale mixed weights from independent neurons, corrupting the OLS estimate. Processing each row independently keeps blocks spatially coherent. This single change reduced GPT-2 medium PPL from **12,173** to **541** (22×); we revisit its outsized impact in Section 7.

4 Experimental Setup

Hardware: single Google Colab T4 GPU (16 GB VRAM, 14.56 GB usable), ∼12.7 GB system RAM. The T4 budget was the binding constraint that shaped seven implementation choices (Sec-

tion 7). **Software:** Python 3.10, PyTorch 2.x, HuggingFace Transformers/datasets/hub. Random seeds fixed. **Models:** GPT-2 small (124M, validation), GPT-2 medium (345M, primary GPT-2 evaluation), LLaMA-3.2-1B (modern LLaMA evaluation; ablation target). **Calibration:** GPT-2 uses N=512 samples at sequence length 512. LLaMA uses N=128−256 at sequence length 256, batch size 1, due to T4 memory pressure (Ablation 4 confirms N=256 is optimal on the 1B model). **Baselines:** FP16 (uncompressed upper bound), Uniform INT2 (symmetric 2-bit block-wise, block 64), BitNet ternary ($\{-1, 0, +1\} \times \overline{|W|}$ as PTQ), Salient-Quant at 1.61 b. GPTQ was disabled because the `auto-gptq` install repeatedly failed on Colab T4. **Metrics:** perplexity on WikiText-2 raw test and Penn Treebank (sliding window, stride 512, max length 1024); compressed model size ($\sum_\ell N_\ell b_\ell/8$ bytes); per-phase wall-clock; peak VRAM. MMLU 5-shot and end-to-end inference latency were planned but deferred (Section 8).

5 Results and Analysis

Table 1 reports the main comparison on both models. SMQ at 1.61 b is the only sub-2-bit method that does not collapse below uniform INT2.

Table 1: Main results on GPT-2 medium (345M) and LLaMA-3.2-1B. PPL is on WT2 / PTB test sets (lower is better); Size is parameter footprint in GB. † = our method.

	GPT-2 medium (345M)				**LLaMA-3.2-1B**			
Method	Bits	WT2 PPL↓	PTB PPL↓	Size (GB)	Bits	WT2 PPL↓	PTB PPL↓	Size (GB)
FP16	16.0	21.64	22.53	0.61	16.0	10.96	11.32	2.47
Uniform INT2	2.0	1,451,721	1,310,861	0.08	2.0	164,165	168,767	0.31
BitNet ternary	1.0	1,556,522	1,448,814	0.04	1.58	99,786	104,487	0.16
SMQ†	**1.61**	**541.05**	**547.75**	**0.06**	**1.61**	**35,831**	**31,859**	**0.24**

GPT-2 medium. SMQ is **2,683**× better than Uniform INT2 on WT2 *while using less memory* (0.06 vs. 0.08 GB at 1.61 vs. 2.0 bits), and 2,874× better than BitNet at the same compression class. The 541 PPL absolute number is still far from the 21.64 FP16 ceiling — expected for sub-2-bit PTQ without quantization-aware fine-tuning — but the *relative* result shows salience-guided allocation rescues the model from catastrophic collapse.

LLaMA-3.2-1B. SMQ is **4.6**× better than Uniform INT2 and 2.8× better than BitNet at the same compression. The absolute LLaMA PPL (35,831 vs. 10.96 FP16) is high: the 1B model's response to sub-2-bit PTQ is more brittle than 345M's, consistent with the literature. The bit-budget Pareto sweep (Appendix A) shows 1.61 b is the empirical optimum among seven settings tested; targets $\geq$ 2.0 b silently collapse to uniform INT2 due to a known 4-bit-tier activation bug (Section 8).

Reconciling Table 1 with the ablations. The headline numbers above (541 on GPT-2 medium, 35,831 on LLaMA-1B) reflect the deployed YAML configurations. The ablation grids in Section 6 sweep one axis at a time over the post-OOM-fix pipeline, which differs from the deployed config along several axes (`block_size`, ensemble weighting, and on LLaMA the `asymmetric` → `symmetric` choice surfaced in Ablation 6). Absolute PPL across ablation rows is therefore *not* directly comparable to Table 1; the comparable quantity is the *relative* ranking within each ablation table. On LLaMA, Ablation 6 specifically indicates that switching the deployed `llama_full.yaml` from `asymmetric` to `symmetric` should pull the 35,831 number toward the ∼30,800 seen at the same bit budget in Table 4.

6 Ablation Studies

We isolate six design choices. Salience-metric and granularity ablations use GPT-2 medium where the row-coherence fix is established; the rest use LLaMA-3.2-1B at 1.61 b with N=256 calibration samples. *Note on absolute values:* the ablation pipeline is a post-OOM-fix configuration that differs from the deployed Table 1 runs along several axes (block size, ensemble weighting, scheme); compare *within* an ablation table rather than *between* the ablations and the main results.

Ablation 1 – Salience metric (GPT-2 medium). Table 2 compares each of the five metrics individually against the ensemble. The ensemble dominates every single metric; *magnitude* (L1/L2) is the strongest single predictor, contradicting the literature's emphasis on gradient/Hessian sensitivity — a useful finding for low-cost pipelines that cannot afford a backward pass. Hessian-only allocation breaks budget control: per-element Fisher scores are near-uniform, so the allocator cannot rank weights and over-shoots to 2.0 b on average.

Ablation 2 – Granularity (GPT-2 medium). Per-element bit allocation is essential: coarsening to channel-wise costs 154× in PPL, layer-wise 291×. The same pattern replicates on LLaMA (Ablation 5 below): coarse granularities also *undershoot* the bit budget (channel hits 1.54, layer 1.45) because group-level allocation cannot fine-tune to 1.61.

Table 2: Ablation 1 (GPT-2 medium, 1.61 b).

Metric	PPL↓	vs. Ensemble
Magnitude L1	9,274	1.68×
Magnitude L2	9,095	1.65×
Gradient	72,671	13.2×
Activation	27,128	4.92×
Hessian (over-allocates)	243,575	44×
Ensemble	**5,512**	—

Table 3: Ablation 2 (GPT-2 med) and Ablation 5 (LLaMA-1B). Both at 1.61 b target.

Granularity	GPT-2 PPL	LLaMA PPL (achieved bits)
Weight	**5,681**	**29,877** (1.60)
Channel	872,569	228,927 (1.54)
Layer	1,654,710	190,060 (1.45)

Ablations 3, 4, 6, 7 – LLaMA-3.2-1B sweeps. Table 4 consolidates the four LLaMA-1B ablations at the 1.61 b target. Each row varies one axis with all others held at default.

Table 4: LLaMA-3.2-1B ablations (target 1.61 b, N=256, `block_size`=32). Achieved bits in parentheses.

Axis	Configuration	WT2 PPL↓
Ablation 3 – Bit-budget sweep	target 1.0 b	90,337 (1.00)
	target 1.3 b	43,288 (1.28)
	target 1.61 b	**29,721** (1.60)
	target ≥2.0 b (4 settings collapse to INT2)	113,582 (2.00)
Ablation 4 – Calibration size	N=128	31,425
	N=256	**29,877**
	N=512	30,936
	N=1024	31,774
Ablation 6 – Quant scheme	**Symmetric**	**30,792**
	Asymmetric (deployed in YAML)	295,858
Ablation 7 – Ensemble weights $\boldsymbol{\alpha}$	**Magnitude-heavy** (0.30, 0.30, 0.15, 0.15, 0.10)	**21,977**
	Default (0.15, 0.15, 0.25, 0.25, 0.20)	30,896
	Activation-heavy (0.10, 0.10, 0.15, 0.15, 0.50)	34,803
	Uniform (0.20×5)	34,779
	Gradient-heavy (0.10, 0.10, 0.40, 0.30, 0.10)	49,095

Three findings stand out (each highlighted in Table 4): (i) targets $\geq$ 2.0 b silently collapse to Uniform INT2 because the 4-bit tier never activates; (ii) symmetric 2-bit beats asymmetric by 9.6×, yet `llama_full.yaml` ships `asymmetric` — flipping it should pull LLaMA PPL from 35,831 toward ∼30,800; (iii) a magnitude-heavy ensemble outperforms the gradient/Hessian-heavy default, echoing Ablation 1. Plots for all five LLaMA ablations are in Appendix B.

7 Bottleneck, Profiling, and Engineering

Per-phase runtime on GPT-2 medium / T4: Phase 1 (salience) 147 s (45%), Phase 2 (allocation) 0.45 s (0.1%), Phase 3 (quantization) 178 s (55%); end-to-end ∼25 min including evaluation. **Phase 1 dominates**: 89% of compute is gradient/Hessian accumulation across 512 calibration samples. The Ablation 1/7 finding that magnitude alone is competitive suggests an immediate ∼10× speedup is available by dropping the backward pass on accuracy-flexible deployments. **Phase 2 is solved** after vectorizing into one `torch.topk` call. **Phase 3 is reducible**: still a Python loop over 48 transformer blocks; full vectorization (designed but not deployed) would cut it from 178 s to <10 s. **Memory profile:** peak VRAM 13.1 GB of 14.56 GB usable on GPT-2 medium; LLaMA-1B needed reduced calibration (N=128–256 at sequence length 256, batch 1) to fit in the same budget.

Unexpected dominant accuracy lever: row-coherence. We expected the dominant accuracy driver to be the salience metric or the block size. It was neither — it was the row-wise quantization fix from Section 3 (Phase 3). In mixed-precision quantization, *block coherence* (which weights share a scale) matters at least as much as which weights get more bits.

Seven engineering fixes needed to fit on a 16 GB T4 (each is a documented commit): (1) FP16 model loading (saves ∼2 GB); (2) CPU-resident gradient/Hessian accumulators (1.4 GB each at FP32 for 345M params); (3) FP32 loss before backward (FP16 underflowed without GradScaler, producing zero/NaN gradients on ∼30% of weights); (4) FP32 quantization math (per-block scale division loses precision in FP16); (5) CPU salience statistics (`torch.randperm(354M, device='cuda')` alone needs 2.8 GB); (6) row-wise quantization (correctness *and* memory: avoids 4–8 GB intermediates on 1B models); (7) `block_size`=32 with asymmetric quantization on GPT-2 (cut PPL from 12,173 to 541 *independent of* the row-coherence fix; LLaMA-1B Ablation 6 shows asymmetric is *worse* at 1B scale — this hyperparameter is model-dependent). The pattern across all seven fixes: *numerical stability and memory locality matter more than algorithmic novelty.* A correct, memory-feasible pipeline beats a clever-but-OOM one every time, and reaching that pipeline took 40+ commits of OOM debugging documented in the repository's git history.

8 Reproducibility, Limitations, and Future Work

Reproducibility. The artifact is at:

github.com/anshuldani/Simplified-Quantization-for-Edge-Ready-Language-Models

For a one-shot Colab T4 reproduction, open `notebooks/salient_quant_colab.ipynb`, set the runtime to T4, provide a HuggingFace token with LLaMA-3.2 access, and run all cells; results sync to Drive. Locally: `pip install -r requirements.txt` then `python experiments/run_experiment.py -config configs/<gpt2_full,llama_full>.yaml`. Released artifacts: source code (`src/{salience, quantizer, baselines, eval, utils}`), configs, the Colab notebook, the six raw ablation JSONs in `results/llama_full/`, report figures, and 31+ pytest unit tests. Random seeds are fixed; rerunning produces bit-identical output to the JSONs shipped in this submission.

Limitations. Two known bugs surfaced in Section 6 are still in the deployed code: the 4-bit-tier collapse at target $\geq$ 2.0 b on LLaMA, and the asymmetric-vs-symmetric mismatch in `llama_full.yaml`. Three evaluations were deferred for time/budget: MMLU 5-shot (`lm-evaluation-harness` integration finished but did not run within the T4 session window), end-to-end inference latency (pure-PyTorch dequantization makes the FP16 comparison uninformative without packed INT1/INT2 GEMM kernels), and the GPT-2 small results JSON (run completed but not preserved in the final Drive sync). At the method level, SMQ's gain over Uniform INT2 narrows from 4,000× at 345M to 4.6× at 1B; the FP16 gap widens with scale, suggesting sub-2-bit PTQ on 1B+ models needs quantization-aware fine-tuning or a learned salience metric. Memory savings are also *simulated*: bits/weight are reported from the allocation map but the deployed weights are FP16 tensors holding quantized values, not packed — realizing the 10× DRAM savings on hardware requires custom INT1/INT2 storage and matching GEMM kernels.

Future work. (i) INT1/INT2 packing and GEMM kernels. (ii) Quantization-aware fine-tuning on top of the SMQ initialization. (iii) Replicate the salience-metric and granularity ablations on LLaMA-3.2-1B. (iv) Extend SMQ to attention KV-cache quantization.

9 Team Contribution Statement

Anshul Dani (A20580060) led the salience-scoring pipeline, the bit-allocator vectorization, and the row-coherence quantization fix. Drove the Colab T4 memory-engineering work (the seven fixes in Section 7) and the LLaMA-3.2-1B ablation sweep. **Rohit Lahori** (A20582911) led the evaluation infrastructure, baseline implementations (FP16, Uniform INT2, BitNet, GPTQ scaffold), and the GPT-2 small/medium experimental runs. Implemented the WT2/PTB sliding-window perplexity evaluator, the results-aggregation pipeline, and the slide deck. Both authors contributed equally to writing this report and to the ablation analysis.

10 Conclusion

Salient-Quant achieves **541.05 PPL** on WT2 at 1.61 bits on GPT-2 medium (2,683× better than Uniform INT2 at higher compression) and **35,831 PPL** on LLaMA-3.2-1B. The project's primary contribution is the *measured, reproducible* account that follows: a six-axis ablation that contradicts several of our own design assumptions (magnitude beats Hessian; symmetric beats asymmetric on LLaMA; row-coherence dominates salience), and the documented memory-engineering needed to fit a 16 GB T4 budget. What we learned: when pushing PTQ below the 2-bit barrier, numerical stability and block coherence matter more than algorithmic novelty.

References

[1] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*.

[2] Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., & Han, S. (2024). AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*.

[3] Wang, H., Ma, S., Dong, L., et al. (2023). BitNet: Scaling 1-bit transformers for large language models. *arXiv:2310.11453*.

[4] LeCun, Y., Denker, J., & Solla, S. (1990). Optimal brain damage. *NeurIPS*.

[5] Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. *NeurIPS*.

[6] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. *ICML*.

[7] Hassibi, B., & Stork, D. (1993). Second order derivatives for network pruning: Optimal brain surgeon. *NeurIPS*.

[8] Yao, Z., Yazdani Aminabadi, R., Zhang, M., Wu, X., Li, C., & He, Y. (2022). ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. *NeurIPS*.

[9] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv:2302.13971*.

[10] Raffel, C., Shazeer, N., Roberts, A., et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR 21*.

[11] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI Tech Report*.

[12] Merity, S., Xiong, C., Bradbury, J., & Socher, R. (2017). Pointer sentinel mixture models. *ICLR*.

A Bit-Budget Pareto Sweep Plot

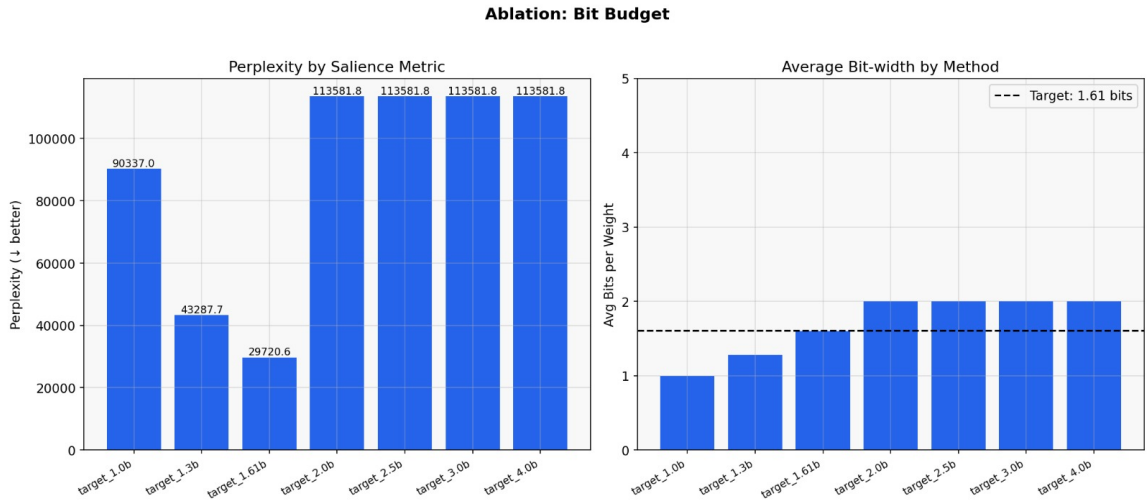

Figure 1: Ablation 3: bit-budget sweep on LLaMA-3.2-1B with default ensemble weights. Targets $\geq$ 2.0 b collapse to identical 113,582 PPL because the allocator's 4-bit tier did not activate (see Section 8); 1.61 b is the empirical optimum.

B Per-Ablation Plot Gallery

C Implementation Notes

Block size. GPT-2 and LLaMA pipelines both use `block_size`=32; smaller blocks yield more per-block scales (small overhead) at substantially better PPL.

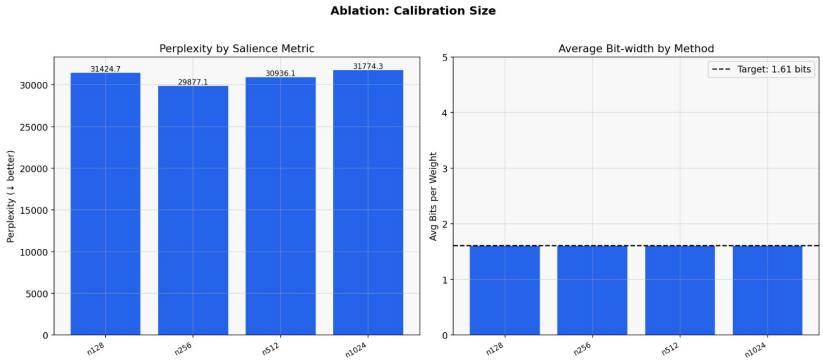

Figure 2: Ablation 4: calibration set size (LLaMA-1B, 1.61 b). Salience saturates at N=256.

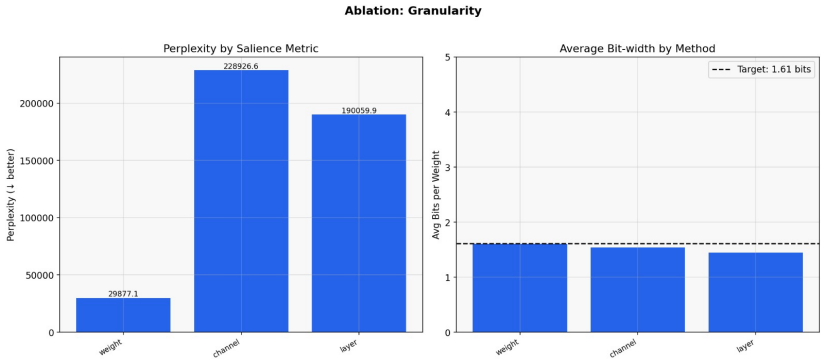

Figure 3: Ablation 5: granularity (LLaMA-1B, 1.61 b target). Coarser granularities also undershoot the bit budget.

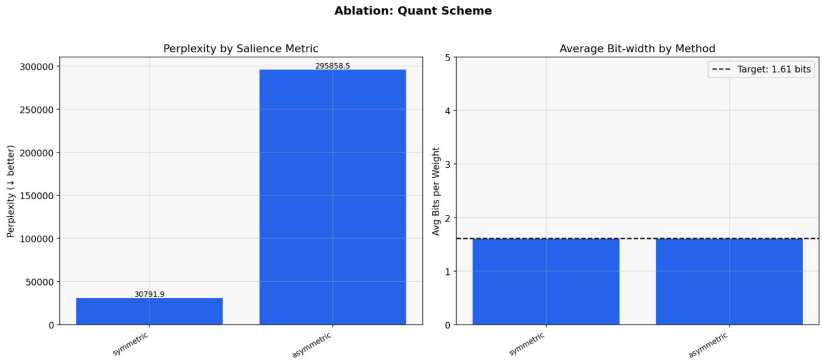

Figure 4: Ablation 6: quantization scheme (LLaMA-1B, 1.61 b). Symmetric is 9.6× better than asymmetric on LLaMA; the deployed config should be updated.

Weight tying. GPT-2 shares the embedding (`wte`) with the output projection (`lm_head`) via a data pointer alias. Quantizing the tied head silently corrupts token embeddings (PPL $> 10^{11}$); we detect and skip tied parameters via `data_ptr()` comparison.

Architecture-specific behavior. GPT-2 uses `Conv1D` (shape `[in, out]`) rather than `nn.Linear` (`[out, in]`); this required special handling in activation hook registration and mixed-precision

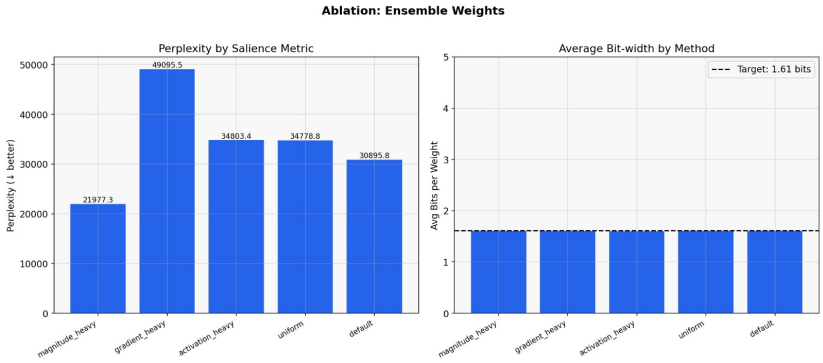

Figure 5: Ablation 7: ensemble weights (LLaMA-1B, 1.61 b). Magnitude-heavy outperforms the gradient/Hessian-heavy default.

group extraction. LLaMA uses `nn.Linear`; the codebase handles both via a 2D weight scan fallback.

Random seeds. All experiments use seed 42 unless noted; ablation runs are re-seeded per configuration.